



Data mining and Data Lec.7.

Genetic Algorithms (GA)



What Are Genetic Algorithms (GAs)?

- It is a type of machine-learning technique that mimics the biological process of evolution

Genetic Algorithms are *search* and *optimization* techniques based on Darwin's Principle of *Natural Selection*.

Basic Idea Of Principle Of Natural Selection

“Select The Best, Discard The Rest”

An Example of Natural Selection

- Giraffes have long necks.
- Giraffes with slightly longer necks could feed on leaves of higher branches when all lower ones had been eaten off.
 - ☐ They had a better chance of survival.
 - ☐ Favorable characteristics propagated through generations of giraffes.
 - ☐ Now, evolved species have long necks.
- NOTE: Longer necks may have been a mutation initially but since it was favorable, it was propagated over generations. Now an established trait.
- So, some mutations are beneficial.

Evolution Through Natural Selection

Initial Population Of Animals



Struggle For Existence-Survival Of the Fittest



Surviving Individuals Reproduce, Propagate Favorable Characteristics



Evolved Species

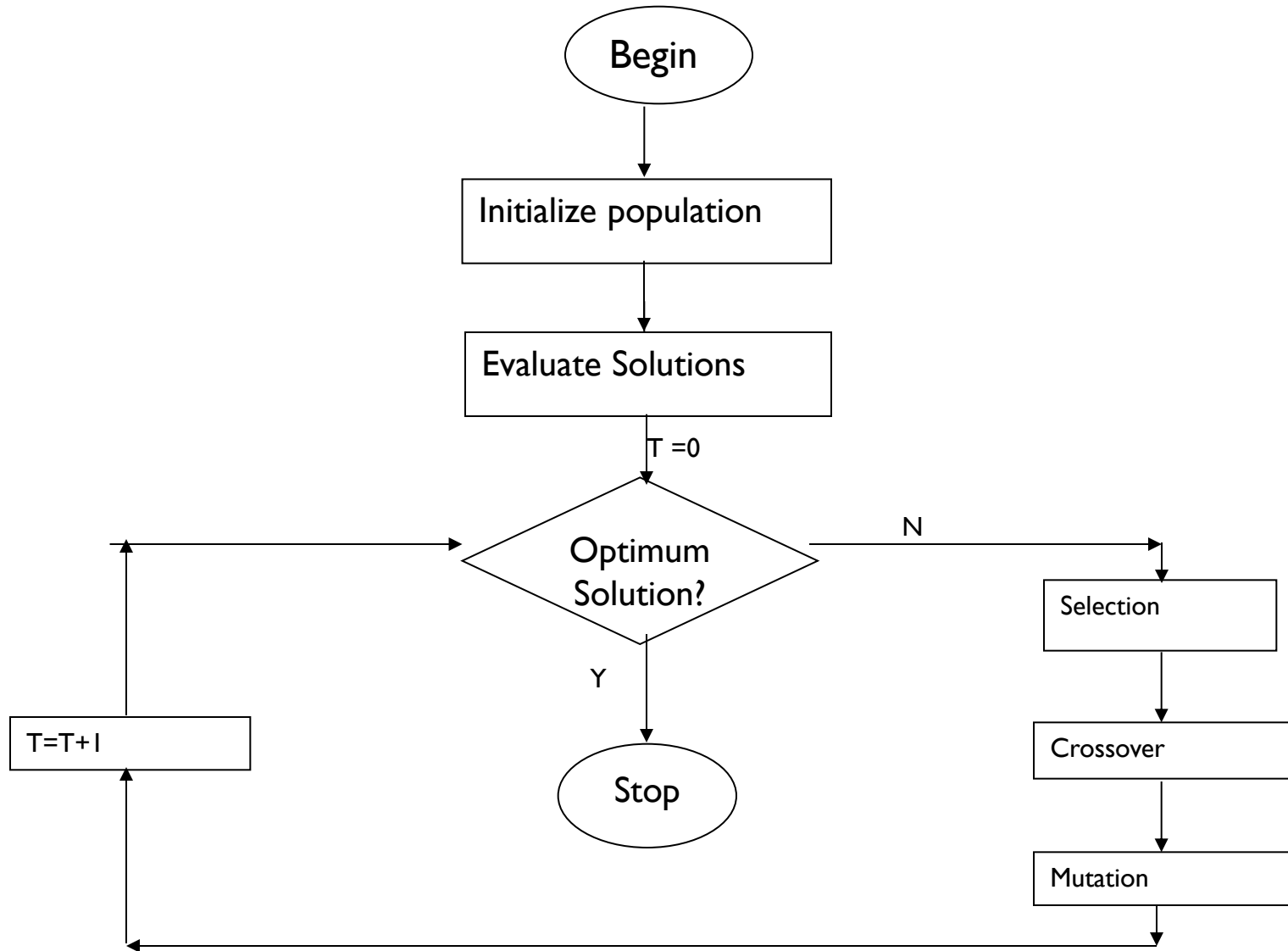
(Favorable Characteristic Now A Trait Of Species)



Millions Of Years

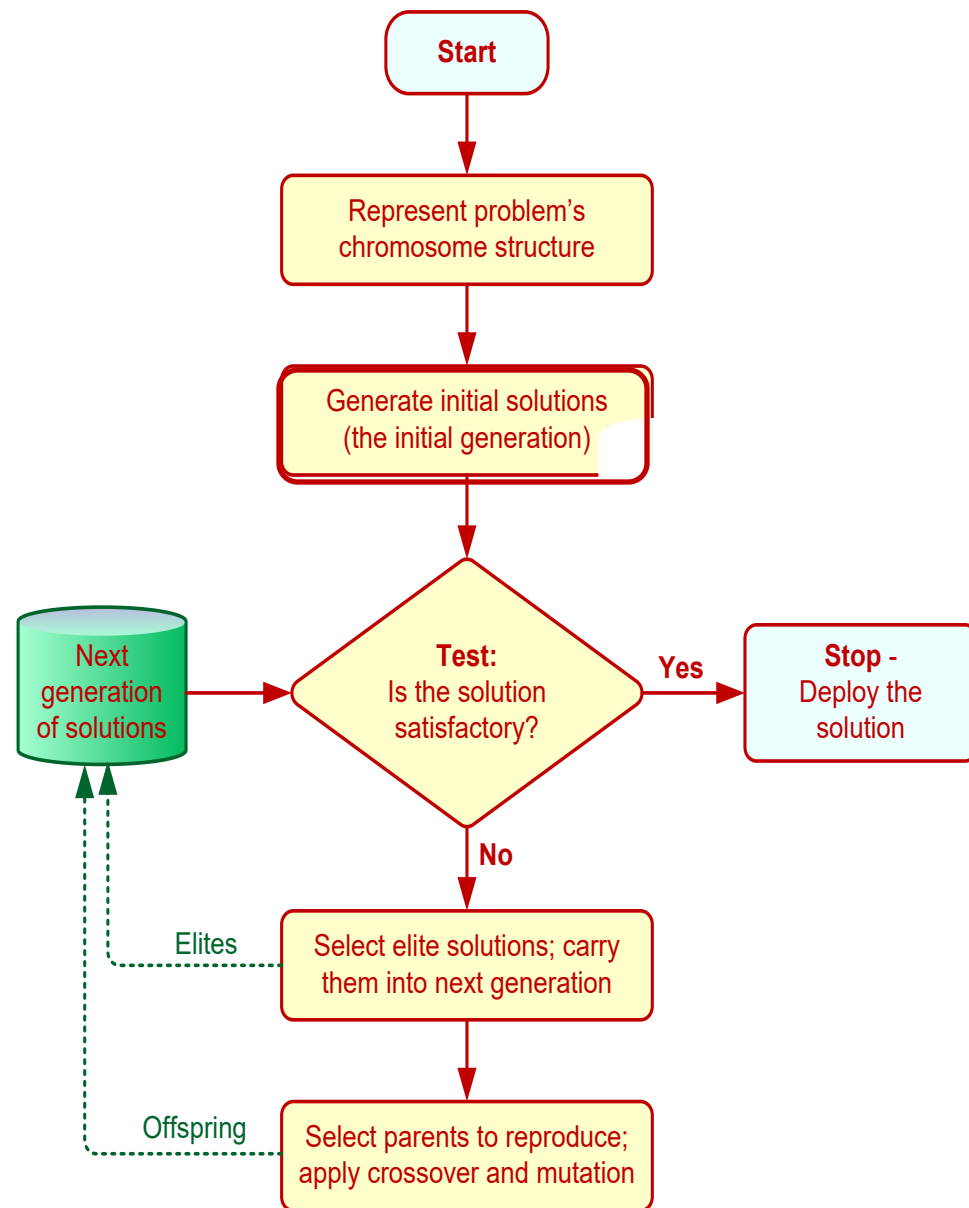
Genetic Algorithms Implement
Optimization Strategies By Simulating
Evolution Of Species Through
Natural Selection.

Working Mechanism Of GAs



GA Structure and GA Operators

- Each candidate solution is called a **chromosome**
- A chromosome is a string of **genes**
- Chromosomes can copy themselves, mate, and mutate via evolution
- In GA we use specific **genetic operators**
 - **Reproduction**
 - Crossover
 - Mutation



GA Requirements

- A typical genetic algorithm requires two things to be defined:
 - a **genetic representation** of the solution domain, and
 - a **fitness function** to evaluate the solution domain.
- The main property that makes these genetic representations convenient is that their parts are easily aligned due to their fixed size, that facilitates simple crossover operation.
- Variable length representations may also be used, but crossover implementation is more complex in this case.
- Tree-like representations are explored in Genetic programming.

GA Requirements

- The fitness function is defined over the genetic representation and measures the quality of the represented solution.
- The fitness function is always problem dependent.
- A representation of a solution might be an array of bits, where each bit represents a different object, and the value of the bit (0 or 1) represents whether or not the object is available.

Representation

Chromosomes could be:

- Bit strings (0101 ... 1100)
- Real numbers (43.2 -33.1 ... 0.0 89.2)
- Permutations of element (E11 E3 E7 ... E1 E15)
- Lists of rules (R1 R2 R3 ... R22 R23)
- Program elements (genetic programming)
- ... any data structure ...

Genetic Algorithm (GA) Steps:

(1) Initialize a population (P) of n elements.

(Randomly generate solutions or "chromosomes" for the problem).

(2) Until a specified termination condition is satisfied (perform iterations).

(2_a) Determine the fitness for all the solutions in the population.

(The **fitness of a solution** is a measure that can be used to compare solutions to determine which is better. **For example**, a company is trying to maximize a profit. The profit itself can be used as the fitness).

(2_b) (**Selection**) Creation of a mating pool, Use a fitness function to evaluate each element of the current population. If an element passes the fitness criteria, it remains in P.

(2_c) (**Crossover**). Select randomly internal points (**crossing sites**) of the solutions, then swap the solution parts that follow these points. randomly selecting a pair at a time.

(2_d) (**mutation**). randomly select a small portion of the solutions and artificially change it (e.g., a bit of 1 to 0 or 0 to 1).

Genetic Learning Operators:

- Apply an evolutionary approach to inductive learning (based on Darwinian principle of natural selection).
- Genetic algorithm can be developed for supervised and unsupervised data mining.
- **Crossover:** forms new elements from the population by combining parts of two elements currently in the population.
- **Mutation:** can be applied by randomly flipping bits within a single elements.

- **Example:**

The secret number is 001010

step.1.

(A) 110100; score = 1 (one digit correctly guessed)

(B) 111101; score = 1

(C) 011011; score = 4

(D) 101100; score = 3

- **Because none of the strings is entirely correct, continue.**

Step 2:

- **Delete (A) and (B) because of their low scores. Call (C) and (D) parents.**

- Step 3:
"Mate" the parents by splitting each number as shown between the second and third digits (the position of the split is randomly selected):

(C) 01.1011

(D) 10.1100



- Step 3:
"Mate" the parents by splitting each number as shown between the second and third digits (the position of the split is randomly selected):

(C) 01.1011

(D) 10.1100

Now combine the first two digits of (C) with the last four of (D) (this is called *crossover*). The result is (E), the first offspring:

(E) 011100; score = 3

001010

combine the first two digits of (D) with the last four of (C). The result is (F), the second offspring:

(F) 101011; score = 4

- Step 4:

Now copy the original (C) and (D).

Step 5:

Mate and crossover the new parents, but use a different split. Now you have two new offspring, (G) and (H):

(C) 0110:11; score = 4

(D) 1011:00; score = 3

(G) 0110:00; score = 4

(H) 1011:11; score = 3

Next, repeat Step 2: Select the best "couple" from all the previous solutions (E,F,G,H) to reproduce.

- Here are the results:

(F) 1:01011

(G) 0:11000

(I) 111000; score = 3

(J) 001011; score = 5

You can also generate more offspring:

(F) 101:011

(G) 011:000

(K) 101000; score = 4

(L) 011011; score = 4

Now repeat the processes with (J) and (K) as parents, and duplicate the crossover:

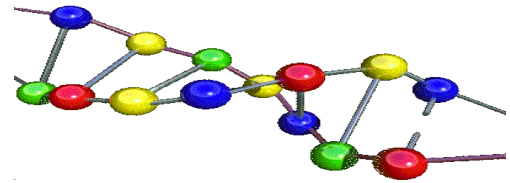
(J) 00101:1

(K) 10100:0

(M) 001010; score = 6

You have reached the solution after 13 guesses.

Genetic Algorithms



- Limitations of Genetic Algorithms

- Does not guarantee an optimal solution (often settles in a sub optimal solution / local minimum)
- Not all problems can be put into GA formulation
- Development and interpretation of GA solutions requires both programming and statistical skills
- Relies heavily on the random number generators
- Locating good variables for a particular problem and obtaining the data for the variables is difficult
- Selecting methods by which to evolve the system requires experimentation and experience

Genetic Algorithm Applications

- Dynamic process control
- Optimization of induction rules
- Discovery of new connectivity topologies (NNs)
- Simulation of biological models of behavior
- Complex design of engineering structures
- Pattern recognition
- Scheduling, transportation and routing
- Layout and circuit design
- Telecommunication, graph-based problems